

In theory, if you have great test coverage, then your project is more maintainable because developers can find out if their changes affected the rest of the system by simply running their unit tests. The tricky thing about code coverage is that it tells you how much of your codebase is exercised by tests, not how good your tests are. If you write a test that runs a method but doesn't assert anything to see if the result is the expected one, then that method is considered covered, even though the test doesn't check anything. There are two things you can do to your code coverage metric to add more value:

- ***Mutation testing*** —Comparing test results with test results after the underlying code has changed or mutated
- A*dding data points to show the value of coverage*—Using data from PTS, SCM, and CI

Mutation testing is an automated test for your tests. Essentially it messes with your code before your unit tests run. This way, if your tests pass after inserting blatant errors, you know that even though a test is executing your